BLOCKCHAIN-BASED DISTRIBUTED DATA LAYER AND PROOF-OF-VERIFICATION LEDGER FOR SMARTGRAMA

INDIVIDUAL THESIS

Dissanayaka D M Y R

(IT22145048)

Individual component: Blockchain-Based Distributed Data Layer & Inter-Bank Verification System

Group ID: R26-IT-017

Report submitted in partial fulfilment of the requirements for the

B.Sc. (Hons) Degree in Information Technology

Specialised in Information Technology

Department of Information Technology

Sri Lanka Institute of Information Technology

Sri Lanka

October 2026

DECLARATION AND SUPERVISOR CERTIFICATION

Candidate declaration. I declare that this thesis is my own work and does not incorporate, without acknowledgement, material previously submitted for a degree or diploma at any university or institute of higher learning. To the best of my knowledge, it contains no material previously published or written by another person except where acknowledgement is made in the text. I grant the Sri Lanka Institute of Information Technology the non-exclusive right to reproduce and distribute this thesis, in whole or in part, in print, electronic, or other media, while retaining the right to use the content in future work.

Academic integrity and AI disclosure. I have disclosed all use of generative AI and other automated tools in accordance with the module policy. I remain responsible for the accuracy, originality, citations, code, data, analyses, and conclusions in this thesis.

Student name: Dissanayaka D M Y R

Student ID: IT22145048

Signature: _____ Date: _____

Supervisor certification. The above candidate carried out this research for the bachelor's degree thesis under my supervision.

Supervisor: Dr. Darshana Kasthurirathna Signature: _____ Date: _____

Co-supervisor: Ms. Chathurya Kumarapperuma Signature: _____ Date: _____

ABSTRACT

Microfinance and welfare administration depend on reliable records and accountable verification by authorized officers. Where institutions maintain separate databases and manual workflows, it can be difficult to establish whether a record has been reviewed, when verification occurred, and which authorized actor performed it. This thesis presents the blockchain component of the SmartGrama platform: a permissioned distributed data layer for inter-institution loan verification and a Proof-of-Verification audit model for administrative verification events. The design uses Hyperledger Fabric, two development organisations, a shared channel and JavaScript chaincode. The loan-record model includes loanId, NIC, loanAmount, loanType, status and timestamp, while detailed financial attributes are assigned to off-chain storage. The Proof-of-Verification event model identifies the authorized officer, verification type, event time, target record reference and transaction evidence without requiring detailed personal documents to be placed on the shared ledger. The implementation artefacts include the Fabric test-network configuration, chaincode design and lifecycle work, component diagrams, and a test catalogue. During development, a proposal-response mismatch associated with timestamp differences was observed; therefore, the results distinguish network and chaincode preparation from a confirmed end-to-end committed transaction. The thesis concludes that a permissioned ledger is a suitable architectural approach for shared tamper-evident verification records, while privacy controls, deterministic transaction behaviour and reproducible commit/query evidence remain essential acceptance criteria for any operational deployment.

Keywords: Hyperledger Fabric, blockchain, inter-bank verification, hybrid data storage, chaincode

ACKNOWLEDGEMENTS

I would like to express my sincere gratitude to my research supervisor, Dr. Darshana Kasthurirathna, and co-supervisor, Ms. Chathurya Kumarapperuma, for their guidance and academic feedback. I also acknowledge the academic environment and technical documentation that supported the study of Hyperledger Fabric and distributed ledger systems. I thank my project colleagues for the wider SmartGrama context that helped define the blockchain component's requirements, and my family for their encouragement throughout the research process

Table of Contents

DECLARATION AND SUPERVISOR CERTIFICATION 2

ABSTRACT 3

ACKNOWLEDGEMENTS 4

LIST OF FIGURES 10

LIST OF TABLES 11

LIST OF ABBREVIATIONS 12

1 INTRODUCTION 13

1.1 Overall project context 13

1.2 Individual component context 13

1.3 Problem statement 13

1.4 Research gap 13

1.5 Aim and objectives 14

1.6 Research questions 14

1.7 Scope, limitations and assumptions 14

1.8 Individual contribution 14

1.9 Thesis structure 15

1.10 Operational context and motivation 15

1.11 Terminology and boundaries 16

1.12 Research significance 17

2 LITERATURE REVIEW AND RELATED WORK 17

2.1 Review method and scope 17

2.3 Related research 18

2.4 Existing systems, products and techniques 19

2.5 Gap synthesis and conceptual framework 19

2.6 Permissioned versus public blockchain 19
2.7 Consensus, endorsement and transaction lifecycle 20
2.8 Data minimisation and privacy by design 20
2.9 Related-system comparison framework 21
2.10 Synthesis of literature and design implications 22
3 RESEARCH METHODOLOGY 22
3.1 Research design 22
3.2 Data, participants or study materials 22
3.3 Experimental or study procedure 23
3.4 Measures and evaluation metrics 23
3.5 Data analysis 23
3.6 Ethics, privacy, security and responsible AI 23
3.7 Reproducibility and research artefacts 24
3.8 Design-science approach and artefact evaluation 24
3.9 Variables, measures and acceptance criteria 24
3.10 Data handling and ethical safeguards 25
3.11 Threats to validity 26
4 PRODUCT AND SOLUTION DESIGN 26
4.1 Requirements 26
4.2 Component architecture within the complete solution 27
4.3 Design alternatives and decisions 27
4.4 Detailed design 27
4.5 Standards and professional practice 28
4.6 Detailed on-chain and off-chain data dictionary 29
4.7 Data validation and canonicalisation 29

4.8 Access-control and endorsement policy 30
4.9 Architecture alternatives and decision rationale 31
4.10 Error handling and recovery design 31
5 IMPLEMENTATION, INTEGRATION AND TESTING 32
5.1 Development environment and technology stack 32
5.2 Component implementation 32
5.3 Integration with the other components 33
5.4 Verification and software testing 33
5.5 Deployment and operation 35
6 RESULTS AND VALIDATION 35
6.1 Research results 35
6.2 Product and technical results 35
6.3 User or stakeholder validation 35
6.4 Integration results 36
6.5 Summary of findings 36
6.6 Evidence classification 36
6.7 Requirements traceability 37
6.8 What the results do and do not establish 37
7 DISCUSSION 38
7.1 Interpretation of findings 38
7.2 Comparison with literature and existing solutions 38
7.3 Research contribution 38
7.4 Product contribution 39
7.5 Limitations and threats to validity 39
7.6 Implications 39

7.7 Interpreting the hybrid data model 39
7.8 Organizational trust and governance 40
7.9 Comparison with non-blockchain alternatives 41
7.10 Implications for future evaluation 41
8 COMMERCIALISATION AND SUSTAINABILITY 42
8.1 Value proposition and target users 42
8.2 Market and competitive position 42
8.3 Business or sustainability model 42
8.4 Scalability, intellectual property and compliance 42
8.5 Contribution to the Sustainable Development Goals 43
8.6 Operational cost categories 43
8.7 Governance and service operations 43
8.8 Regulatory and responsible deployment considerations 44
9 CONCLUSION AND FUTURE DIRECTIONS 44
9.1 Conclusion 44
9.2 Achievement of objectives 45
9.3 Individual research and product contributions 45
9.4 Future directions 45
9.5 Final synthesis 46
9.6 Prioritised implementation roadmap 46
REFERENCES 47
GLOSSARY 47
APPENDICES 48
Appendix A: Research instruments and extended results 48
Appendix B: Academic integrity and AI use disclosure 48

Appendix C: Evidence interpretation and evaluation boundaries 48
Appendix D: Previously Shared Implementation 49
Screenshots 49
Appendix E: Detailed Requirements Traceability Matrix 51
Appendix F: Security and Privacy Threat Model 52
Appendix G: Data Dictionary and Example Record 54
Appendix H: Reproducibility Runbook 55
H.1 Confirm the working directory 55
H.2 Set Fabric paths 55
H.3 Start the network 56
H.4 Configure Org1 56
H.5 Configure Org2 56
H.6 Package and install 56
H.7 Approve and commit 56
H.8 Execute a synthetic test 56
H.9 Verify final commitment 56
H.10 Record failures 56
H.11 Clean up 57
Appendix J: Chaincode Design Listing and Review Notes 57
Appendix K: Architecture and Implementation Figures 58

LIST OF FIGURES

Figure 1 49
Figure 2 50
Figure 3 59
Figure 4 60
Figure 5 61
Figure 6 62

LIST OF TABLES

Table 1 34
Table 2 51
Table 3 52
Table 4 54

LIST OF ABBREVIATIONS

API — Application Programming Interface

CA — Certificate Authority

DLT — Distributed Ledger Technology

MSP — Membership Service Provider

NIC — National Identity Card

PII — Personally Identifiable Information

SHA-256 — Secure Hash Algorithm 256-bit

TLS — Transport Layer Security

UAT — User Acceptance Testing

1 INTRODUCTION

1.1 Overall project context

Digital financial services depend on accurate and timely information about applicants and their existing obligations. When financial institutions operate separate application systems and databases, information available to one institution may not be immediately visible to another. A distributed verification layer can provide a common record of selected events without requiring every institution to place all customer information in one central database. This individual component sits within a wider SmartGrama/FinanceEase project context.

1.2 Individual component context

The component, titled “Blockchain-Based Distributed Data Layer & Inter-Bank Verification System”, uses Hyperledger Fabric as a permissioned distributed ledger platform. Org1MSP and Org2MSP represent two participating organizations in the development network. The intended application flow is frontend to backend/API, then to an off-chain store and Fabric. The frontend should not directly access Fabric peers or hold administrator credentials.

1.3 Problem statement

Where lenders rely on institution-specific databases, another lender may have difficulty establishing whether an applicant already has a loan elsewhere. This fragmented view can cause repeated data collection, slower verification, inconsistent records and incomplete assessments. A central shared database could reduce fragmentation but concentrates operational control. The research problem is how to design a shared, auditable verification layer while limiting the sensitive customer information exposed through the shared ledger.

1.4 Research gap

Blockchain provides shared transaction histories and programmable rules, but does not automatically solve privacy, data quality or governance. A practical design must identify which fields should be shared, which should remain private, how duplicate records are handled and how multiple organizations participate in validation. This work addresses that engineering gap through a permissioned-ledger prototype and hybrid storage model. It is not a complete credit-risk assessment system.

1.5 Aim and objectives

Aim: To design and implement a permissioned blockchain-based data layer that supports auditable inter-bank loan verification while keeping detailed financial information off-chain.

1. Design a minimal on-chain loan-verification record.
2. Separate shared metadata from detailed off-chain financial information.
3. Configure a Hyperledger Fabric test network with two organizations.
4. Implement chaincode to create loan records and reject duplicate loan identifiers.
5. Define the integration boundary between frontend, backend, database and Fabric.
6. Evaluate the prototype using functional tests and documented technical evidence.

1.6 Research questions

7. How can a permissioned blockchain support shared verification of selected loan records between independent institutions?
8. Which loan-related data should be on-chain and which should remain off-chain?
9. How can chaincode enforce basic record-integrity rules, including duplicate rejection?
10. What technical and operational limitations remain before production use?

1.7 Scope, limitations and assumptions

The scope includes the ledger layer, loan-record model, chaincode lifecycle and intended backend integration. The on-chain fields are loanId, NIC, loanAmount, loanType, status and timestamp. The first four are transaction inputs; status and timestamp are chaincode-managed in the intended design. Off-chain fields are monthly_income, other_income, expenses, savings_balance, existing_loans, repayment_history and guarantor_support_count. The prototype uses a local test network, does not prove real-bank adoption, does not calculate credit eligibility, and does not automate a final lending decision.

1.8 Individual contribution

The individual contribution focuses on the distributed verification component: defining the minimal ledger record, separating on-chain and off-chain fields, configuring a two-organization Fabric development network, preparing chaincode lifecycle operations, and documenting the backend integration boundary. The final submission should include only code and results that the student personally implemented or can demonstrate.

1.9 Thesis structure

Chapter 2 reviews technical foundations and related approaches. Chapter 3 explains the methodology. Chapter 4 presents requirements and design. Chapter 5 describes implementation and testing. Chapter 6 reports evidence and validation. Chapter 7 discusses findings and limitations. Chapter 8 considers sustainability and commercialisation. Chapter 9 concludes the thesis and identifies future work.

1.10 Operational context and motivation

Digital lending workflows depend on the ability to identify an applicant, collect a consistent application record, and assess existing financial obligations. A lender may hold accurate information about its own customer while having only partial visibility of borrowing at other institutions. This is not necessarily a failure of an individual database: it is a boundary problem between separately governed information systems. A shared verification mechanism must

therefore address the need for cross-institution evidence without assuming that every institution should disclose its complete customer database to every other participant.

The proposed component investigates a narrow technical response to that boundary problem. It records a small set of loan-verification metadata on a permissioned ledger and leaves detailed financial information in an off-chain store. This choice is deliberate. A distributed ledger is most useful here as a common, auditable coordination layer; it is not intended to replace every bank's operational database, credit policy, customer relationship system, or legally required recordkeeping process.

The development prototype is evaluated as a software artefact rather than as a deployed financial service. A local Fabric test network can demonstrate chaincode packaging, organizational approval, transaction endorsement, ordering, commitment and query behaviour. It cannot, by itself, establish that multiple real banks are ready to operate a consortium network, that all source data is correct, or that the design satisfies every legal and regulatory obligation. The thesis separates these claims so that the value of the prototype is not overstated.

1.11 Terminology and boundaries

In this thesis, an institution or bank is represented by a Fabric organization. Org1MSP and Org2MSP are development identities used to demonstrate organizational separation; they are not evidence that two real banks have enrolled. A peer is a node that hosts ledger data and may execute chaincode for endorsement. The ordering service establishes a common transaction order. A channel is a logical communication and ledger boundary shared by authorized organizations. Chaincode is the executable business logic used to validate and update ledger state.

The term on-chain refers to state and transaction evidence maintained by the Fabric ledger. Off-chain refers to information held outside that ledger, such as a database record or document store. A transaction hash or application-generated SHA-256 digest must not be confused with a transaction identifier or a committed block. A digest can support integrity checking when its input and comparison process are defined, but hashing alone neither encrypts the source data nor proves that a transaction was committed to Fabric.

The component boundary is intentionally limited to inter-bank loan verification metadata. It does not implement a complete credit scoring engine, autonomous loan approval, AI fraud detection, or a national identity verification service. A final lending decision remains with an authorized institution and its established procedures.

1.12 Research significance

The technical significance of the work is the combination of a permissioned distributed ledger, a minimal record schema, duplicate-key protection, and an explicit off-chain data boundary. Each design decision is individually familiar; the research value lies in specifying how these decisions work together for a narrowly defined verification workflow and identifying the conditions required for a trustworthy implementation.

The practical significance is conditional. If institutions agree on the meaning of each shared field, the permitted queries, the endorsement policy, the correction process, and the governance of membership, a shared ledger may reduce ambiguity in the evidence exchanged between participants. If those agreements do not exist, a blockchain can merely replicate inconsistent data. Consequently, governance and data quality are treated as design requirements rather than as benefits automatically supplied by the technology.

2 LITERATURE REVIEW AND RELATED WORK

2.1 Review method and scope

This review focuses on authoritative technical documentation and standards relevant to permissioned blockchain, chaincode, ledger state, private data and cryptographic hashing. It is a focused technical review rather than a claim of a systematic review of all loan-verification research. Add peer-reviewed studies on blockchain in financial services and credit-information systems before final submission

2.2 Domain and theoretical foundations

2.2.1 Distributed ledger technology

A distributed ledger maintains a replicated record across multiple network participants. Participants follow agreed protocols for validating and ordering updates rather than relying

exclusively on one database administrator. This can improve shared auditability, but adds coordination and governance complexity

2.2.2 Hyperledger Fabric

Hyperledger Fabric is an enterprise-oriented permissioned blockchain framework. Its architecture includes peers, an ordering service, channels, membership identities, policies and chaincode. Fabric separates transaction simulation and endorsement from ordering and commitment, and organizations can define endorsement policies [1], [2]. This supports a prototype in which participating institutions have distinct administrative identities.

2.2.3 Smart contracts and chaincode

A smart contract defines the business logic that reads or updates ledger state. In Fabric, chaincode is packaged, installed on peers, approved by organizations and committed to a channel. A transaction must satisfy endorsement and validation rules [2]. In this project, the principal invariant is that a loan identifier must not already exist before a new record is created.

2.2.4 Cryptographic hashing

A cryptographic hash maps input data to a fixed-length digest. SHA-256 is specified in NIST FIPS 180-4 and can help detect changes to data [3]. Hashing does not encrypt the original information, prove that information was true, or by itself prove a blockchain transaction was committed. Therefore, application-generated hashes must not be described as blockchain evidence without ledger verification.

2.2.5 Hybrid storage

Hybrid storage places a small set of verification fields or references on-chain while detailed records remain in a conventional database or protected document store. This reduces unnecessary replication of personal information. However, a hash or reference can still be linkable to an individual, especially when combined with a NIC. Data minimisation, authorization and retention controls remain necessary.

2.3 Related research

Blockchain prototypes commonly explore shared records, audit trails and multi-party workflows. The key design lesson is to use a ledger for the coordination problem it solves: a jointly

verifiable record of a loan event. It should not replace every institution's operational database. A ledger can show that a record was committed, but cannot independently confirm that declared income or repayment history is accurate.

2.4 Existing systems, products and techniques

A centralized shared database offers straightforward querying but relies on a central operator. Bilateral APIs preserve local control but may create many integrations and inconsistent audit trails. A permissioned ledger provides a shared transaction history, while hybrid storage keeps detailed financial records off-chain. These options have trade-offs in privacy, cost, performance and governance; blockchain is not automatically superior for every use case.

2.5 Gap synthesis and conceptual framework

The gap addressed is the combination of a multi-organization ledger, a minimal shared loan record, duplicate-identifier protection and separate storage for detailed financial data. A participating institution submits a record through an authorized application; the backend validates the request; detailed fields remain off-chain; chaincode records verification metadata; and another authorized institution can query the record under agreed policies. This supports auditability rather than automated lending decisions.

2.6 Permissioned versus public blockchain

Public blockchain networks generally allow broad participation in transaction submission and validation, subject to protocol rules. This openness can be valuable for systems that do not rely on a known membership set, but it may introduce performance, confidentiality, fee and governance considerations that are poorly matched to regulated inter-organizational financial workflows. A lender-to-lender verification network is more likely to require authenticated membership, explicit roles, and agreed operating policies.

A permissioned network uses identity credentials and governance rules to constrain participation. Hyperledger Fabric follows this model through membership service providers, certificates, channel configuration, access control and endorsement policies. Permissioned does not mean automatically private in every respect: organizations must still configure channel membership, transaction visibility, network access, certificate handling, and off-chain storage securely. The

trust model is shifted from a single database operator to a governed set of known participants; it is not eliminated.

For this component, Fabric is appropriate as a research platform because it makes organizational identity and transaction endorsement visible in the development workflow. The choice does not imply that Fabric is universally superior to centralized databases. The decision depends on whether several institutions need a common state history while retaining independent administrative control.

2.7 Consensus, endorsement and transaction lifecycle

Fabric separates transaction simulation and endorsement from ordering and validation. An endorsing peer simulates chaincode against its current state and returns a proposal response containing a read-write set and response. Required organizations endorse according to the chaincode definition and policy. The ordering service orders endorsed transactions into blocks. Peers then validate transactions against endorsement and versioning rules before committing blocks to their ledgers.

This lifecycle matters to the prototype because a successful command response at one stage is not equivalent to final ledger commitment. Package creation proves that a chaincode package was prepared. Installation proves that a peer accepted a package. Approval and commit establish a channel-level chaincode definition. An invoke proposal may still fail endorsement or validation. Successful final verification should therefore include the transaction result and a subsequent query of the committed state.

Endorsement also creates a determination of determinism. If two endorsing peers execute the same transaction and produce different read-write sets, their proposal responses cannot be combined into one valid transaction. In the development history, timestamp serialization produced a proposal-response mismatch. This is evidence that generated values must be designed and tested for consistent endorsement. It should be discussed as an observed implementation risk, not hidden by presenting a later run as successful without proof.

2.8 Data minimisation and privacy by design

Financial records can contain direct identifiers, income details, liabilities, repayment behaviour, household circumstances and supporting documents. Replicating all of these fields to every peer

increases the number of systems that must be secured and complicates correction and retention. A privacy-oriented architecture begins by asking what another institution needs to know to perform the verification task, rather than copying the entire source record.

The selected on-chain schema is loanId, NIC, loanAmount, loanType, status and timestamp. The user-provided fields are loanId, NIC, loanAmount and loanType; status and timestamp are intended to be generated by chaincode. Detailed financial fields remain off-chain: monthly_income, other_income, expenses, savings_balance, existing_loans, repayment_history and guarantor_support_count. The current design still includes NIC as an on-chain field, which is a privacy risk and must be treated as a prototype choice requiring further review. A production design should evaluate a stable pseudonymous reference, access-controlled lookup, private data collections, or another privacy-preserving identifier before exposing raw NIC values to a shared ledger.

Data minimisation does not mean that the remaining metadata is harmless. Loan amount and loan type may still reveal sensitive financial information. A permissioned network needs access control, justified membership, audit trails, secure communications, key management, and documented retention and correction processes. Off-chain storage also needs encryption, least-privilege access, backups, and a reliable way to bind a database record to the ledger record.

2.9 Related-system comparison framework

A centralized shared database provides a simple authoritative query endpoint and may be appropriate where one trusted operator is acceptable. Its risks include concentration of control, dependence on the central service, and the need for every participant to trust the operator's audit and change-management practices. These risks can be mitigated with replication and strong governance; they should not be caricatured as inevitable failures.

Bilateral API integrations preserve each institution's local data ownership, but the number of interfaces can grow as more institutions join. Each connection requires authentication, authorization, schema mapping, monitoring and change coordination. A shared ledger can reduce the number of independently negotiated data paths by establishing a common transaction and record model, but it adds ledger operations, chaincode lifecycle management, consortium governance and endorsement-policy complexity.

A conventional credit bureau or regulated shared-credit service may already provide a legally recognized verification function. The proposed prototype should not be presented as a replacement for such services without comparative evidence, regulatory analysis and stakeholder agreement. The comparison in this thesis is architectural: it explains the trade-offs that motivate a permissioned prototype and identifies the questions that a future feasibility study must answer.

2.10 Synthesis of literature and design implications

The reviewed concepts lead to six design implications. First, organizational identity must be explicit. Second, chaincode should enforce a small number of deterministic business rules. Third, sensitive detailed data should not be replicated by default. Fourth, transaction success must be demonstrated through ledger commitment and query evidence. Fifth, hashing must be described accurately as an integrity mechanism rather than as encryption or proof of authorization. Sixth, governance and data correction must be designed alongside the technology.

These implications are reflected in the requirements and test plan. They also define the limitations of the work. The prototype can demonstrate technical feasibility in a local network, but real-world usefulness depends on common data definitions, legal authority to share information, consent or another valid processing basis where applicable, operational service levels, and agreement on how disputes are resolved.

3 RESEARCH METHODOLOGY

3.1 Research design

The work follows a design-and-development approach. A software artefact is designed, implemented in a controlled environment and assessed through functional and integration tests. The unit of analysis is the loan-verification workflow and its ledger state transitions, not individual borrowers or real financial outcomes.

3.2 Data, participants or study materials

Development should use synthetic demonstration loan records. Real customer financial information should not be used without an approved purpose, appropriate lawful basis and safeguards. Study materials include chaincode source, Fabric configuration, CLI output,

application request/response evidence and test cases. This draft does not claim testing with real banks or production customer data

3.3 Experimental or study procedure

1. Define the loan schema and classify on-chain and off-chain fields.
2. Start the Fabric test network and verify both peer organizations and the ordering service.
3. Package chaincode and record its label, version and sequence.
4. Install chaincode, approve the definition for each organization, check readiness and commit.
5. Submit a valid loan-creation transaction and query the ledger state.
6. Attempt a duplicate loanId and verify rejection.
7. Test malformed inputs, authorization failures and network failures where implemented.
8. 18.Capture sanitized logs, transaction identifiers and screenshots

3.4 Measures and evaluation metrics

Functional correctness is assessed by whether valid records are created, duplicates are rejected and queries return expected values. Ledger evidence requires a committed transaction and a query of its state, not merely a frontend message or generated hash. Integration is assessed by whether the backend can submit requests without exposing Fabric credentials to the browser. Privacy is assessed by checking whether detailed financial fields are absent from the public ledger. Performance should be reported only after repeated measurements.

3.5 Data analysis

Test outcomes should be classified as pass, fail or not tested. Logs and transaction responses are reviewed against expected behavior. Any quantitative performance study should report the number of trials, workload, environment, mean, median and range. No numerical performance improvement is claimed in this draft because verified measurements were not supplied.

3.6 Ethics, privacy, security and responsible AI

A NIC is a sensitive identifier and must not be treated as harmless because it is on a permissioned network. Production use requires purpose limitation, least-privilege access, transport encryption, protected secrets, logging, retention controls and secure off-chain storage.

A blockchain’s durable history creates challenges for correction and deletion, so incorrect records need a governance process. Generative-AI assistance used for this thesis must be disclosed under module policy; the student remains responsible for all claims and citations.

3.7 Reproducibility and research artefacts

Reproducibility artefacts should include chaincode source, package label and sequence, Fabric version, network configuration, commands, test data and sanitized outputs. Private keys, enrollment secrets, database credentials and real personal data must not be included.

3.8 Design-science approach and artefact evaluation

The research follows a design-and-development approach: a problem is identified, objectives are defined, an artefact is designed and implemented, and its behaviour is evaluated against explicit requirements. The artefact is the blockchain data-layer prototype, including the record schema, chaincode, Fabric test-network configuration and integration boundary. The evaluation is primarily functional and technical; it is not a clinical, financial-outcome or population-level study.

The evaluation strategy distinguishes verification from validation. Verification asks whether the implementation conforms to its specification—for example, whether a duplicate loan identifier is rejected. Validation asks whether the specification is useful for the intended institutional workflow—for example, whether the chosen shared fields are sufficient for an authorized lender to verify an existing loan. The first can be tested in a development environment. The second requires domain experts, authorized data and a defined evaluation protocol, none of which should be claimed without evidence.

Each test is recorded with an identifier, preconditions, steps, expected result, observed result, and evidence reference. Where a result was not captured, the correct classification is 'not demonstrated in the available implementation record' or 'not tested', rather than pass. This approach prevents successful network setup from being used as indirect proof of all application-level requirements.

3.9 Variables, measures and acceptance criteria

The primary functional measures are: successful creation of a valid unique record; rejection of a duplicate identifier; correct retrieval of a committed record; consistent endorsement across

required peers; and graceful reporting of network or validation errors. Security-oriented checks include validation of required fields, avoidance of secret disclosure in logs, and confirmation that only configured identities can perform permitted actions. Not all security checks are implemented in the current prototype, so the evaluation matrix identifies them as not executed in the recorded evaluation where appropriate.

Performance measures could include transaction submission latency, time to final commitment, query latency, throughput under a defined workload, and error rate. These metrics require a repeatable test environment, specified hardware and software versions, a defined number of repetitions, and a workload description. No performance figures are invented in this thesis. A future benchmark should report medians and percentiles as well as averages, because transaction latency can vary under container startup, resource contention and network conditions.

The minimum acceptance criterion for a claimed ledger write is not merely a frontend confirmation. It is a transaction accepted by the Fabric lifecycle and a subsequent query that returns the expected committed state. For a multi-organization endorsement test, the evidence should identify the participating peers and show that the proposal response was consistent. A failed or mismatched endorsement is recorded as a failed test and investigated.

3.10 Data handling and ethical safeguards

Demonstration data should be synthetic and clearly marked as such. The prototype does not need real NICs, real account balances, or real repayment histories to demonstrate the chaincode lifecycle. Using fabricated identifiers reduces the risk of exposing personal information in screenshots, terminal histories, logs and thesis appendices.

If real personal data were ever considered, the research team would need to establish an approved purpose, lawful authority, participant or institutional permissions where required, data minimisation, access controls, retention periods and a response plan for incidents. Screenshots should be reviewed for visible credentials, certificates, private keys, database connection strings, access tokens and personally identifying details before being included in a report.

Ethical handling also includes accurately reporting limitations. A prototype should not imply that a customer is creditworthy, ineligible, fraudulent or approved merely because a record exists on a ledger. The ledger can establish that a particular record was submitted and committed under the

network's rules; the meaning and correctness of the underlying data still depend on authorized source systems and institutional processes.

3.11 Threats to validity

Construct validity may be affected if a successful UI event is used as a proxy for a committed blockchain transaction. The mitigation is to use transaction and query evidence. Internal validity may be affected by inconsistent timestamps, stale network state, package-version confusion, or configuration differences between shells. These should be controlled through a recorded environment and repeatable command sequence.

External validity is limited because a local test network does not reproduce independent institutions, production security operations, real network latency, real user volumes or regulatory governance. Reliability is limited if tests are performed once without saved outputs. The thesis therefore describes the implementation as a prototype and identifies the evidence needed for stronger claims.

Researcher bias can arise when failed commands are omitted from the narrative. A reproducible report should preserve meaningful failure evidence, explain the diagnosis, and distinguish resolved issues from outstanding blockers. The timestamp endorsement mismatch and peer-connectivity problems are particularly relevant examples because they show why successful packaging or a running Docker container is not equivalent to a completed end-to-end transaction.

4 PRODUCT AND SOLUTION DESIGN

4.1 Requirements

4.1.1 Functional requirements

- FR1: Accept loanId, NIC, loanAmount and loanType.
- FR2: Reject creation when loanId already exists.
- FR3: Assign status through chaincode.
- FR4: Use timestamp handling that produces consistent endorsement results.
- FR5: Allow authorized clients to query committed records.
- FR6: Keep detailed financial attributes in an off-chain store.

4.1.2 Non-functional requirements

- Security: only authorized identities may submit or query protected records.
- Integrity: transactions satisfy endorsement and validation rules.
- Privacy: minimize personal data replicated to channel peers.
- Maintainability: separate contract, API and database responsibilities.
- Reliability: handle network errors and do not show success before commit confirmation.
- Auditability: retain transaction identifiers and relevant logs.

4.2 Component architecture within the complete solution

The architecture has four logical layers: frontend, backend/API, off-chain database and Hyperledger Fabric network. The frontend collects an application and displays results. The backend validates inputs, enforces authorization, coordinates the database and calls Fabric. The off-chain store holds detailed financial attributes. Fabric stores the minimal shared record. The frontend must not contain peer administrator certificates or directly connect to Fabric.

The architecture is illustrated by the conceptual and component flow diagrams in Appendix K. The frontend communicates with the backend/API, which coordinates the off-chain data store and Hyperledger Fabric. The Fabric network contains the ordering service and peers representing Org1 and Org2 on the shared channel. Diagram labels distinguish the component boundary from functions that belong to the wider SmartGrama platform.

4.3 Design alternatives and decisions

A centralized database is simpler but requires participants to depend on a central operator. Bilateral APIs preserve institutional databases but can increase integration complexity. A permissioned ledger was selected for the prototype because it supports shared state transitions, organizational identities and configurable endorsement policies [1], [2]. Detailed financial fields are kept off-chain to reduce unnecessary exposure and replication.

4.4 Detailed design

4.4.1 On-chain data model

- loanId — unique loan record key.
- nic — applicant identifier; requires strict protection and review before production.

- loanAmount — requested or recorded amount using a consistent validated type.
- loanType — controlled loan category.
- status — chaincode-managed state, such as ACTIVE in the prototype.
- timestamp — transaction-related time associated with record creation.

4.4.2 Off-chain data model

The off-chain attributes are monthly_income, other_income, expenses, savings_balance, existing_loans, repayment_history and guarantor_support_count. The backend should associate them with loanId or a secure internal reference and enforce access control and consistency.

4.4.3 Transaction flow

11. User submits an application through the frontend.
12. Backend validates fields and caller authorization.
13. Backend stores detailed financial fields in the protected off-chain database.
14. Backend submits CreateLoan to Fabric.
15. Chaincode checks whether loanId already exists and rejects duplicates.
16. Endorsing peers simulate the transaction; Fabric validates and commits it if valid.
17. Backend returns success only after commit confirmation and records the transaction ID.
18. An authorized institution queries the record and follows its own lending procedures.

4.4.4 Determinism and timestamp design

All endorsing peers must produce matching proposal results. Values that differ between endorsers can cause proposal-response mismatch. Timestamp generation and serialization must therefore be tested with the deployed Fabric version. If the client supplies the timestamp, the thesis must say so. If Fabric transaction time is used, the implementation must be tested to confirm matching state. Do not claim deterministic behavior without evidence.

4.5 Standards and professional practice

The design draws on Fabric's documented transaction, identity and chaincode lifecycle model [1], [2] and NIST's Secure Hash Standard [3]. Professional practice includes secure secret management, least-privilege access, input validation, version control, reproducible builds and honest reporting. Applicable Sri Lankan data-protection and financial-sector requirements must be assessed before using real customer data.

4.6 Detailed on-chain and off-chain data dictionary

The ledger record contains six fields. loanId is the record key and must be unique within the chaincode's chosen namespace. NIC is the applicant identifier used by the current prototype to associate a record with a person; its use requires a privacy review and should not be interpreted as a verified identity unless an independent identity-verification process exists. loanAmount is the amount associated with the loan record and should be represented consistently, preferably as a validated decimal string or integer minor units rather than a floating-point value. loanType identifies the institution's agreed category. status is generated by chaincode according to the current contract rule. timestamp is intended to describe transaction time and must be implemented deterministically for endorsement.

The off-chain record contains monthly_income, other_income, expenses, savings_balance, existing_loans, repayment_history and guarantor_support_count. These fields can support downstream financial assessment, but they are not required to be replicated on the ledger for the narrow verification purpose. The off-chain database should store a reference to loanId and should enforce its own schema validation, access control and audit policy. If a digest of the off-chain record is stored on-chain, the exact canonical representation, hash algorithm, versioning and re-hashing procedure must be documented; otherwise two semantically identical records can produce different hashes due to ordering, whitespace or number formatting.

The split is a design boundary, not a complete privacy guarantee. A lender querying a ledger record may still infer information from loanAmount, loanType, status and timing. The production version should evaluate whether all six fields are necessary for every participant, whether the raw NIC can be replaced, and whether some fields belong in a private data collection or a protected off-chain service.

4.7 Data validation and canonicalisation

Validation should occur at the API boundary and in chaincode where the ledger rule depends on it. API validation can provide user-friendly feedback, while chaincode validation is necessary because ledger rules must not rely solely on a frontend being well behaved. Required values should be non-empty, loanId should conform to a documented character and length policy, loanAmount should be positive and within a defined range, and loanType should be drawn from

an agreed list. NIC format checks can detect obvious formatting errors but do not prove that the identifier belongs to the applicant.

Canonicalisation is important for repeatable results. Amounts should use one representation, identifiers should not silently change through whitespace trimming or case conversion unless the rules specify it, and JSON fields should be serialized consistently. Chaincode should reject invalid data with a clear error instead of accepting it and leaving another component to infer what was intended.

Chaincode should also avoid local machine time, random values, external network calls or other peer-specific inputs during transaction simulation. Where a timestamp is required, the implementation must test that the value used by all endorsers is identical. If the transaction timestamp is supplied by the Fabric proposal, the code and Fabric SDK behaviour should be verified in the actual deployment. The observed proposal-response mismatch in the development history means this is a test requirement, not an assumption.

4.8 Access-control and endorsement policy

The endorsement policy determines which organizations must approve the simulated result before a transaction can be ordered. For a demonstration intended to show inter-bank participation, a policy requiring both Org1MSP and Org2MSP provides visible evidence of multi-organization approval. The exact policy must be selected according to the threat model and operational requirements; requiring every organization for every transaction may increase availability risk as membership grows.

Identity and authorization should be considered separately. A valid certificate identifies a network member, but an application may still need to distinguish a bank administrator, loan officer, read-only verifier and service identity. Access control should be enforced through chaincode client-identity checks, Fabric policies, backend authorization, or a combination appropriate to the use case. The prototype's two-organization setup should not be described as full role-based access control unless the relevant rules are implemented and tested.

Membership lifecycle is also part of the design. Production governance must define who may add an organization, revoke compromised credentials, approve chaincode upgrades, change

endorsement policies, restore a peer, and resolve disputes about incorrect records. These decisions cannot be delegated entirely to code.

4.9 Architecture alternatives and decision rationale

Alternative A is a single shared database operated by a central service. It has simple querying and familiar backup and recovery mechanisms, but all participants must trust the operator's access and audit controls. Alternative B is a set of bilateral APIs. It keeps data with each institution but requires point-to-point agreements and schema coordination. Alternative C is a permissioned distributed ledger, which can provide a common ordered history under multi-party governance while adding operational complexity.

The selection criteria are institutional independence, auditability, confidentiality, integration effort, performance, operational cost and governance complexity. Fabric is selected for the prototype because its organizations, peers, chaincode lifecycle and endorsement policies directly support experimentation with independent participants. This is a research choice for the stated problem, not a measured claim that Fabric is faster or cheaper than the alternatives.

A later feasibility study should compare the alternatives using representative workloads and institutional requirements. It should include costs for identity infrastructure, secure hosting, monitoring, incident response, consortium governance, API development and data-quality management. A technically functioning prototype is only one input into that decision.

4.10 Error handling and recovery design

The backend should return clear, bounded error messages and avoid exposing internal stack traces, certificate paths, private data or connection secrets to a browser. The frontend should distinguish validation failure, unavailable service, rejected transaction and confirmed ledger commitment. A success notification should be displayed only after the backend has received the relevant commit status and, for critical workflows, verified the expected state.

Retries require care. Retrying a transaction after a timeout can create uncertainty about whether the first request committed. The application should use a stable idempotency or loan identifier and query the ledger before submitting a duplicate. A duplicate-key rejection can be a useful safeguard, but it is not a complete distributed idempotency design if requests can be accepted through different keys or namespaces.

Recovery procedures should cover a stopped peer, unavailable orderer, expired or revoked certificate, chaincode startup failure, failed endorsement and a database/ledger consistency problem. The application must not silently treat off-chain storage success as ledger success, or vice versa. A reconciliation process should identify records present in only one storage layer and route them for controlled correction.

5 IMPLEMENTATION, INTEGRATION AND TESTING

5.1 Development environment and technology stack

The development environment used Windows with WSL/Ubuntu for Fabric command-line tooling and Docker Desktop for containers. The Fabric test network contains peer0.org1.example.com, peer0.org2.example.com and orderer.example.com, with Org1MSP and Org2MSP participating on mychannel. The recorded peer binary version was Hyperledger Fabric v2.5.15. The chaincode was written in JavaScript for Node.js using fabric-contract-api. The final report should state exact frontend, backend and database versions from the working repository.

5.2 Component implementation

5.2.1 Fabric network

The Fabric test network demonstrates two independent organizational identities. The Org1 peer was configured through localhost:7051 and Org2 through localhost:9051 in the local development environment; the ordering service was exposed through localhost:7050. Network startup, peer readiness, TLS configuration and channel membership are prerequisites for chaincode operations.

5.2.2 Chaincode contract

The JavaScript contract defines a CreateLoan operation. It retrieves state for the requested loanId and rejects the request if a record already exists. Otherwise, it constructs a loan object containing loanId, NIC, loanAmount, loanType, status and timestamp, serializes the object as JSON, and writes it under loanId. Production validation should additionally check missing values, amount format, permitted loan types, NIC format and field lengths.

5.2.3 Chaincode lifecycle

Fabric chaincode lifecycle includes packaging source code, installing the package on required peers, approving the definition for each organization, checking readiness and committing the definition to the channel [2]. Development records indicate successful organizational approvals and chaincode definition commitment for earlier versions. The final thesis must align package label, definition version, sequence number and source with the exact version demonstrated in final evidence.

5.2.4 Backend and off-chain integration

The backend is the trust boundary between browser and infrastructure. It should validate requests, write detailed financial fields to the off-chain store, submit Fabric transactions using a supported client interface, and return success only when the outcome is confirmed. A locally generated SHA-256 digest may support integrity checking, but it is not a substitute for invoking chaincode and verifying ledger state. Describe the actual Gateway/SDK used in the final build; do not claim SDK integration if the backend only uses a separate CLI demonstration.

5.3 Integration with the other components

The distributed layer is intended to integrate with the group application through a backend API. The frontend sends application data to the backend; the backend keeps detailed fields off-chain and calls Fabric for the shared record. The API should define request fields, response status, error formats, transaction ID handling and duplicate behavior. For multi-machine deployment, localhost endpoints must be replaced with reachable addresses and secured with TLS and network access rules.

5.4 Verification and software testing

The following catalogue defines the verification cases used to assess the component. Each case is classified according to the implementation evidence available for this thesis; cases without execution evidence are explicitly described as not executed rather than being presented as successful.

Table 1

ID	Test scenario	Expected outcome	Observed result
TC01	Create loan with a new loanId and valid fields	Record commits and can be queried	Outcome: not established by the recorded test evidence
TC02	Create a record with an existing loanId	Chaincode rejects duplicate	Outcome: not established by the recorded test evidence
TC03	Query a committed loanId	Stored fields are returned	Outcome: not established by the recorded test evidence
TC04	Submit missing required fields	Request is rejected with useful error	Outcome: not established by the recorded test evidence
TC05	Use an unauthorized identity	Request is denied	Outcome: not established by the recorded test evidence
TC06	Disconnect a peer during a request	Failure handled without false success	Outcome: not established by the recorded test evidence
TC07	Inspect public ledger for detailed fields	Off-chain-only fields are absent	Outcome: not established by the recorded test evidence
TC08	Submit with required endorsers	Endorsements match and valid transaction commits	Outcome: not established by the recorded test evidence

A timestamp-related proposal-response mismatch was observed during development when endorsing peers returned slightly different timestamp values. This demonstrates why deterministic chaincode behavior must be tested. The issue is reported as an observed result and a limitation of the evaluated prototype. No correction is claimed without repeatable transaction evidence.

5.5 Deployment and operation

The prototype uses a local Fabric test network and Docker containers. Operational steps include starting the network, checking peer and orderer health, setting the correct organization environment variables, and confirming channel and chaincode status. Include sanitized screenshots of a successful query, duplicate rejection and final network status. Do not include private keys, credentials or sensitive identifiers.

6 RESULTS AND VALIDATION

6.1 Research results

The design outcome is a hybrid model that separates shared verification metadata from detailed financial information. This addresses the question of minimizing unnecessary disclosure while maintaining a common record. Fabric supports shared ledger state and configurable private-data mechanisms [1]. An off-chain database alone does not guarantee privacy; access controls, encryption, retention and secure linkage are also required.

6.2 Product and technical results

Development established a two-organization Fabric test-network configuration and progressed through chaincode packaging, organizational approvals and channel commitment. The CreateLoan design checks for an existing loanId and writes a minimal record for a new identifier. The evaluation distinguishes chaincode preparation from confirmed ledger commitment. The recorded timestamp mismatch is reported as an observed technical result; no unsupported claim of successful multi-endorser commitment is made. Because timestamp-related endorsement mismatch was observed, claim successful multi-endorser execution only after a repeatable test confirms matching proposal responses.

6.3 User or stakeholder validation

No formal evaluation by real banks, loan officers or customers is evidenced in the information available for this draft. Therefore, this thesis does not claim stakeholder acceptance or production readiness. Future validation should use a structured protocol with authorized domain experts and report participant count, questions, consent arrangements and results

6.4 Integration results

The intended integration boundary is frontend → backend/API → off-chain storage and Fabric. A frontend success popup or locally generated transaction hash is not sufficient proof of ledger commitment; the transaction ID and a ledger query should corroborate the result. Include only the integration paths exercised successfully in the final running build.

6.5 Summary of findings

- The hybrid model reduces detailed financial data placed on the shared ledger.
- Fabric provides a permissioned framework for organizational identities, chaincode and shared ledger state [1], [2].
- Duplicate loanId rejection is a functional invariant that can be tested.
- Application-generated hashes must be distinguished from committed blockchain evidence.
- Production readiness requires security hardening, privacy review, governance and broader testing.

6.6 Evidence classification

The available evidence supports several different levels of conclusion. Source-code screenshots support that a chaincode file existed. Package output supports that a package archive was created. Approval and commit outputs support lifecycle progress for the recorded chaincode definition. Docker and health outputs support that certain containers or endpoints were running at the time observed. A successful invoke followed by a query is the stronger evidence needed to establish a committed application record.

These evidence categories must not be collapsed into a single claim of 'blockchain fully working'. In particular, the development record contains a proposal-response mismatch caused by different timestamp values and a later peer connectivity problem during installation of a newer package. Both are relevant results because they expose determinism and environment-integration risks.

The thesis therefore reports implementation progress and known limitations separately. The architecture and data model are design outcomes. Chaincode packaging and channel commitment are lifecycle outcomes. Full end-to-end integration remains unconfirmed unless the final package has a successful transaction and ledger query attached as evidence.

6.7 Requirements traceability

Requirements traceability links each requirement to the design element, implementation element, test case and evidence. For example, the duplicate-key requirement maps to the CreateLoan existence check, the expected rejection test, and the corresponding CLI output. The privacy requirement maps to the hybrid data model and the decision not to store detailed income and repayment fields on-chain. The deterministic endorsement requirement maps to the timestamp implementation and a multi-peer invoke test.

Traceability prevents requirements from being marked complete merely because they are mentioned in the design. A requirement may be Designed, Implemented, Tested or Verified; these statuses should be recorded independently. A designed feature may not be implemented. An implemented feature may not have a saved test result. A test may pass in a local test network without establishing production readiness.

The traceability matrix separates implemented design/code, observed execution outcomes and evaluation cases without execution evidence. This distinction keeps the conclusions aligned with the artefacts and recorded results.

6.8 What the results do and do not establish

The prototype demonstrates the suitability of Fabric concepts for modelling an inter-organization verification record and provides a structure for recording minimal metadata. The recorded lifecycle work demonstrates practical experience with packaging, installation, organizational approval and channel definition commitment. The design also identifies why a backend-mediated architecture and off-chain database are needed for a usable application.

The results do not establish that two real financial institutions have joined the network, that the frontend has successfully submitted a final committed transaction through the backend, that real NIC values have been verified, or that the system is compliant with all applicable laws. They also do not establish a performance advantage over a centralized database or credit-information service. Such claims require evidence beyond the current prototype.

The distinction is important to the research conclusion. A technically honest result that identifies unresolved determinism and connectivity problems is more valuable than a broad claim

unsupported by transaction-level evidence. The future work plan is therefore tied directly to the observed gaps

7 DISCUSSION

7.1 Interpretation of findings

The prototype provides a plausible architecture for sharing limited loan-verification metadata without centralizing all financial information. Its key contribution is the separation of responsibilities: the ledger provides a shared record, chaincode enforces a basic invariant, and the off-chain store retains detailed financial fields. This is a narrower and more defensible use of blockchain than putting the full loan application or lending decision on-chain.

Two organizations demonstrate participation across separate administrative identities. However, two organizations running on one development machine do not prove resilience against real institutional outages, malicious insiders or production-scale traffic. This is a technical prototype, not evidence of adoption by real banks.

7.2 Comparison with literature and existing solutions

Compared with a centralized database, Fabric can distribute the ledger and require transactions to satisfy organizational endorsement rules. Compared with bilateral APIs, a shared channel can provide a common transaction history. These benefits come with complexity around membership, policies, upgrades, governance and incident response [2]. SHA-256 can detect changes to a known input [3], but does not solve identity assurance, authorization or data correctness. Hashing a NIC does not automatically anonymize it.

7.3 Research contribution

The research contribution is a focused architecture for inter-bank loan verification using a permissioned ledger and hybrid data model. It identifies minimal shared information, defines a chaincode duplicate check and describes the organizational lifecycle required to deploy the contract. The contribution is this combination and application context, not the invention of blockchain or hashing.

7.4 Product contribution

The product contribution is a prototype component that can be integrated with a wider application through a backend API. It provides a basis for recording and querying loan-verification metadata. Include repository evidence and screenshots showing which functions work end to end.

7.5 Limitations and threats to validity

- Local test-network deployment is not equivalent to deployment across independent banks.
- A NIC field creates privacy and data-protection risks if shared too broadly.
- Duplicate-key validation does not prove source data is accurate or reveal every liability.
- Timestamp handling can cause endorsement mismatches if endorsers serialize different values.
- No performance or scalability advantage should be claimed without measurements.
- Without formal stakeholder evaluation, usability and institutional acceptance remain unknown.
- Off-chain integrity depends on database controls and reliable linkage to the ledger record.

7.6 Implications

For developers, the work emphasizes deterministic chaincode, minimal shared data and explicit API boundaries. For institutions, it highlights the need for shared governance and clear query/submission rules. For researchers, it provides a basis for empirical testing of privacy, performance and multi-party verification.

7.7 Interpreting the hybrid data model

The hybrid model reflects a principle of minimum necessary disclosure. A lender may need to know that a record associated with a reference exists, what kind of loan is recorded, its amount and status, and when the record was created or updated. It does not automatically need access to the applicant's detailed income, expense breakdown, savings, repayment history or guarantor information. Keeping these fields off-chain reduces replication and supports institution-specific authorization.

However, the selected schema includes NIC directly on-chain. This creates a tension between straightforward lookup and privacy by design. The thesis should be explicit that the current schema is a prototype choice rather than the final production recommendation. A future design could use a keyed pseudonymous identifier, a controlled lookup service, or a Fabric private data collection.

Each option has trade-offs: pseudonyms can still be linkable, private data collections require careful member configuration, and external lookup services introduce availability and governance dependencies.

The design is therefore a starting point for a data-governance discussion. It makes the shared fields visible and reviewable, rather than hiding the disclosure decision inside implementation code.

7.8 Organizational trust and governance

Two organizations on a local development machine demonstrate separate membership identities and policies, but they do not recreate the operational independence of two banks. Both peers may share the same host, administrator, network access and maintenance window. A production consortium would need separate administrative control, secure peer hosting, independent key management, incident response and agreed governance.

Governance should define which organization may submit or correct a record, which organizations must endorse each transaction, what happens when a lender disputes a record, how a mistaken record is amended, how long records remain available, and how membership changes are approved. Because ledger entries are designed to be tamper-evident, correction should generally be represented as a controlled new transaction rather than an undocumented direct edit.

These governance questions influence technical architecture. A strict endorsement policy can increase confidence in shared writes but reduce availability when a required organization is offline. A looser policy may improve availability but require additional controls to prevent one organization from submitting inaccurate information. Policy choice should follow the risk analysis and consortium agreement.

7.9 Comparison with non-blockchain alternatives

The appropriate question is not whether blockchain is more modern than a database. It is whether the participants need a jointly governed, tamper-evident history that no single participant can unilaterally rewrite under the agreed protocol. If one trusted operator is acceptable and audit requirements can be met with conventional controls, a centralized service may be cheaper and simpler. If institutions can establish trusted APIs and a common authority, bilateral or hub-and-spoke integration may be sufficient.

Fabric becomes more attractive when several known organizations require shared state, need to retain their own identities and operational control, and agree to a common transaction process. Even then, total cost includes the ledger platform, membership infrastructure, monitoring, upgrades, security operations and consortium governance. The prototype does not provide measured cost or performance data to settle the alternative selection empirically.

The contribution of the thesis is therefore a design and implementation investigation that clarifies the trade-offs and provides a basis for a future comparative evaluation.

7.10 Implications for future evaluation

A future evaluation should compare the Fabric prototype with a conventional centralized database using the same synthetic dataset, transaction types and hardware. It should measure end-to-end latency, query latency, failure recovery, throughput, operational steps and cost. The test should vary the number of organizations and the endorsement policy, because those factors can influence performance and availability.

Stakeholder evaluation should include authorized loan officers, compliance representatives, security staff and system administrators. Participants should be asked whether the shared fields support the intended decision, whether the query workflow fits their work, what data they consider too sensitive to share, and what evidence they require before relying on another institution's record. Any such study would need appropriate institutional approval and a documented method.

The research should also evaluate false matches and missing records. A ledger can provide a reliable record of what was submitted, but it cannot guarantee that all loans in the financial

ecosystem were submitted. A missing entry must not be interpreted as proof that no loan exists unless participating coverage and data quality are known.

8 COMMERCIALISATION AND SUSTAINABILITY

8.1 Value proposition and target users

Potential target users include banks, licensed finance companies, microfinance providers and other authorized lenders that need to verify selected loan records across institutional boundaries. The value proposition is a shared, auditable mechanism that may reduce repeated manual checks and inconsistencies. These are hypotheses for future validation, not measured business outcomes.

8.2 Market and competitive position

The solution would be compared with centralized credit-information services, direct API integrations and institution-specific workflows. Its differentiator is a permissioned shared ledger governed by multiple organizations. Disadvantages include integration cost, governance complexity and operational overhead. A market assessment requires lender interviews, analysis of existing services, regulatory review and cost-benefit analysis.

8.3 Business or sustainability model

A possible model is a consortium in which participating institutions jointly govern the network and share infrastructure, security and support costs. Costs include hosting, monitoring, certificate management, integration, incident response and audits. Sustainability requires service-level agreements, funding responsibilities, onboarding processes and dispute resolution.

8.4 Scalability, intellectual property and compliance

Adding organizations requires membership onboarding, policy updates, identity lifecycle management, secure connectivity, monitoring and tested upgrades. Before commercial use, the team must review intellectual-property ownership, third-party licences, data-sharing agreements and applicable Sri Lankan data-protection and financial-sector requirements. No compliance certification or production approval is claimed.

8.5 Contribution to the Sustainable Development Goals

The project may indirectly support SDG 9 (Industry, Innovation and Infrastructure) through exploration of digital infrastructure and SDG 16 (Peace, Justice and Strong Institutions) through auditable recordkeeping. These are potential contributions only; the prototype has not measured social impact, financial inclusion outcomes or fraud reduction.

8.6 Operational cost categories

An operational cost model should include infrastructure for peers and ordering services, secure networking, certificates and identity management, monitoring, backup and disaster recovery, chaincode development and upgrades, help-desk support, security testing, and consortium governance. Off-chain storage adds database hosting, encryption, access controls, retention, audit logging and recovery costs. Costs should be allocated transparently among participating institutions.

The number of participants affects the cost profile. More institutions increase the potential coverage of verification, but also increase onboarding effort, policy coordination, certificate lifecycle work, support requirements and the complexity of upgrades. A sustainable model needs a clear mechanism for joining, leaving, susnot executed in the recorded evaluation and restoring members.

No commercial price, saving percentage or return on investment is claimed in this thesis because there is no measured workload or cost baseline. A future business case should compare the proposed system with existing verification processes and include both direct technology costs and the staff time required to resolve exceptions.

8.7 Governance and service operations

A production service needs service-level objectives for availability, transaction confirmation, query response and incident communication. Monitoring should cover peer health, ordering service health, certificate expiry, disk capacity, chaincode errors, transaction rejection rates and off-chain database health. Alerts should have a documented owner and escalation path.

Change management should require review and testing of chaincode updates, policy changes, certificate rotation and network upgrades. The consortium should agree how urgent security

patches are approved and deployed. Backups and disaster recovery tests must include off-chain records and configuration material, while protecting private keys and credentials.

Commercial sustainability depends on trust in operations as much as on the ledger's data model. If members cannot understand how records are submitted, corrected, queried and audited, the technical network may not be adopted even if it functions correctly.

8.8 Regulatory and responsible deployment considerations

Before production deployment in Sri Lanka or another jurisdiction, the consortium would need legal review of personal-data processing, financial confidentiality, credit-information rules, record retention, data subject rights, cross-border hosting and institutional outsourcing obligations. The thesis does not provide a legal opinion and does not claim regulatory approval.

The architecture should support purpose limitation, least privilege, data minimisation and accountability. Raw NIC storage requires particular scrutiny. A production system should also define whether a verification response can be used only as one input to an authorized review or whether any automated decision is permitted. This prototype does not make final lending decisions.

Responsible deployment requires a clear explanation to institutions and customers of what is shared, who can see it, how it is corrected, and how disputes are handled. A tamper-evident record can make a wrong entry durable; controls for data quality and correction are therefore essential

9 CONCLUSION AND FUTURE DIRECTIONS

9.1 Conclusion

This thesis presents a blockchain-based distributed data-layer design for inter-bank loan verification. A permissioned Hyperledger Fabric network maintains a shared record while detailed financial attributes remain off-chain. The chaincode design supports loan-record creation and rejection of duplicate identifiers. The component supports verification and auditability; it does not make final lending decisions or replace institutional risk assessment.

Implementation evidence is essential. A transaction should be considered successful only when committed and verifiable on the ledger. A frontend message or generated SHA-256 digest alone does not prove that a blockchain transaction occurred.

9.2 Achievement of objectives

- Defined a minimal on-chain record and an off-chain financial data model.
- Designed the component around a permissioned Fabric network with two organizations.
- Prepared the CreateLoan chaincode pattern and duplicate-key validation.
- Documented the chaincode lifecycle and intended frontend/backend/ledger boundary.
- Defined functional tests and identified timestamp determinism, privacy and integration as validation priorities.

Revise this list if any item was not completed in the final build. Each achievement must be supported by code, configuration, screenshots or test results.

9.3 Individual research and product contributions

The individual research contribution is the analysis and design of a minimal hybrid data-sharing approach for multi-institution loan verification. The product contribution is the Fabric-based verification component and its chaincode workflow. Shared team components should be acknowledged, and the final thesis should separate individual work from collaborative work.

9.4 Future directions

- Replace raw NIC exposure with a privacy-preserving identifier or controlled reference where feasible.
- Define formal endorsement policies and least-privilege access rules.
- Integrate the backend with Fabric Gateway using secure identity and secret management.
- Use an off-chain database with encryption, audit logging, retention and recovery controls.
- Conduct deterministic transaction, security, load and failure-recovery testing.
- Evaluate the workflow with lending stakeholders and measure verification time and operational cost.
- Add organizations only after governance and privacy controls are established.

9.5 Final synthesis

This work investigates a permissioned distributed data layer for inter-bank loan verification using Hyperledger Fabric. It defines a minimal shared loan record, keeps detailed financial fields off-chain, and uses chaincode to prevent a duplicate loan identifier from overwriting an existing record. It also documents the development lifecycle and the practical distinction between a user-interface success message and a committed ledger transaction.

The main conclusion is conditional: Fabric provides relevant mechanisms for organizational identity, chaincode execution, endorsement and shared ledger state, but trustworthy verification depends on deterministic code, correctly configured policies, reliable source data, privacy controls and clear governance. The observed timestamp mismatch demonstrates that even a small field can prevent endorsement if peer simulations differ. The WSL/Docker connectivity issue demonstrates that deployment configuration is part of implementation evidence, not a minor administrative detail.

The prototype should therefore be treated as a foundation for further validation rather than as a production-ready banking service. The next priorities are to resolve timestamp determinism, complete the newest chaincode lifecycle, obtain successful invoke-and-query evidence, connect the backend through a secure Fabric client, and reconsider raw NIC storage before any real-data pilot.

9.6 Prioritised implementation roadmap

Priority 1 is to establish a reproducible local network baseline with aligned Fabric binaries and container images, explicit TLS configuration, and saved outputs for both peers and the orderer. Priority 2 is to finalize chaincode so that the same proposal produces the same read-write set on every required endorser. Priority 3 is to test valid creation, duplicate rejection, invalid fields, record query and peer failure, saving sanitized evidence for each case.

Priority 4 is to integrate the backend with Fabric Gateway or another supported client. The backend should not report success until commit status is confirmed, and it should use a stable identifier to manage retries. Priority 5 is to implement durable off-chain persistence, access control, encryption, audit logging and reconciliation. Priority 6 is to conduct a privacy and governance review of NIC usage, field visibility, membership and correction procedures.

Only after these priorities are complete should the team consider a controlled pilot with authorized institutional stakeholders. Such a pilot should use synthetic or appropriately approved data, have a defined evaluation protocol, and compare the workflow against existing methods. Performance, adoption and financial benefits should be reported only after measurement.

REFERENCES

[1] Hyperledger Foundation, “Hyperledger Fabric Model,” Hyperledger Fabric Documentation. [Online]. Available: https://hyperledger-fabric.readthedocs.io/en/latest/fabric_model.html. Accessed: Oct. 9, 2026.

[2] Hyperledger Foundation, “How Fabric Networks Are Structured,” Hyperledger Fabric Documentation. [Online]. Available: https://hyperledger-fabric.readthedocs.io/en/latest/network/network.html. Accessed: Oct. 9, 2026.

[3] National Institute of Standards and Technology, “Secure Hash Standard (SHS),” FIPS PUB 180-4, Aug. 2015. doi: 10.6028/NIST.FIPS.180-4.

[4] Hyperledger Foundation, “Private Data,” Hyperledger Fabric Documentation. [Online]. Available: https://hyperledger-fabric.readthedocs.io/en/release-2.4/private-data/private-data.html. Accessed: Oct. 9, 2026.

The reference list records the academic and technical sources cited in this thesis. Reference details should be retained consistently in the required institutional style.

GLOSSARY

Chaincode — Fabric smart-contract code that reads or updates ledger state.

Endorsement policy — Rule defining which organization peers must endorse a transaction.

Hyperledger Fabric — Permissioned distributed-ledger framework for enterprise applications.

Ledger — Blockchain history and current state maintained by Fabric peers.

MSP — Membership Service Provider; associates participants with trusted identities.

Off-chain storage — Data stored outside the blockchain, such as in a protected database.

On-chain data — Data recorded in the ledger and replicated according to network membership.

Peer — Fabric node that hosts a ledger and may execute chaincode.

SHA-256 — A 256-bit cryptographic hash algorithm specified by NIST.

World state — Current values of ledger assets, maintained for efficient queries.

APPENDICES

Appendix A: Research instruments and extended results

Include the final test-case sheet, sanitized CLI outputs, successful transaction identifiers, chaincode query outputs, network topology and any measured performance results. Reference each item from the main thesis. Do not include private keys, passwords, real NIC values or other sensitive information.

Appendix B: Academic integrity and AI use disclosure

Generative AI assistance was used during thesis preparation for outlining, language refinement and document organization. The author is responsible for verifying all technical statements against the implementation artefacts, checking citations, and ensuring that the submitted report accurately reflects the work performed. Any additional tool use should be disclosed in accordance with the applicable SLIIT module policy.

Appendix C: Evidence interpretation and evaluation boundaries

- The included development-environment screenshots show the local Fabric network configuration and available channel/network status information.
- The implementation narrative records the chaincode lifecycle stages and distinguishes package labels from chaincode definition versions and sequence numbers.
- The available screenshots and logs are not treated as proof of a committed CreateLoan transaction unless a successful invoke result and matching post-commit query are both present.
- Duplicate-identifier rejection is defined in the chaincode logic and documented as a required functional behaviour.
- The hybrid data model classifies detailed financial attributes as off-chain and the shared ledger record as a minimal verification record.

- The frontend screenshots demonstrate the interface response only; they are not interpreted as independent proof of ledger commitment.
- The chaincode listing in Appendix J documents the contract pattern discussed in the implementation chapter.

Appendix D: Previously Shared Implementation

Screenshots

The following figures reuse screenshots previously shared during development. They provide supporting evidence of the Fabric development environment, the chaincode source, and the frontend submission flow. A frontend confirmation is not, by itself, proof that a Fabric transaction was committed. Where blockchain commitment is claimed, include a separate successful invoke/query output or transaction evidence from the final working build.

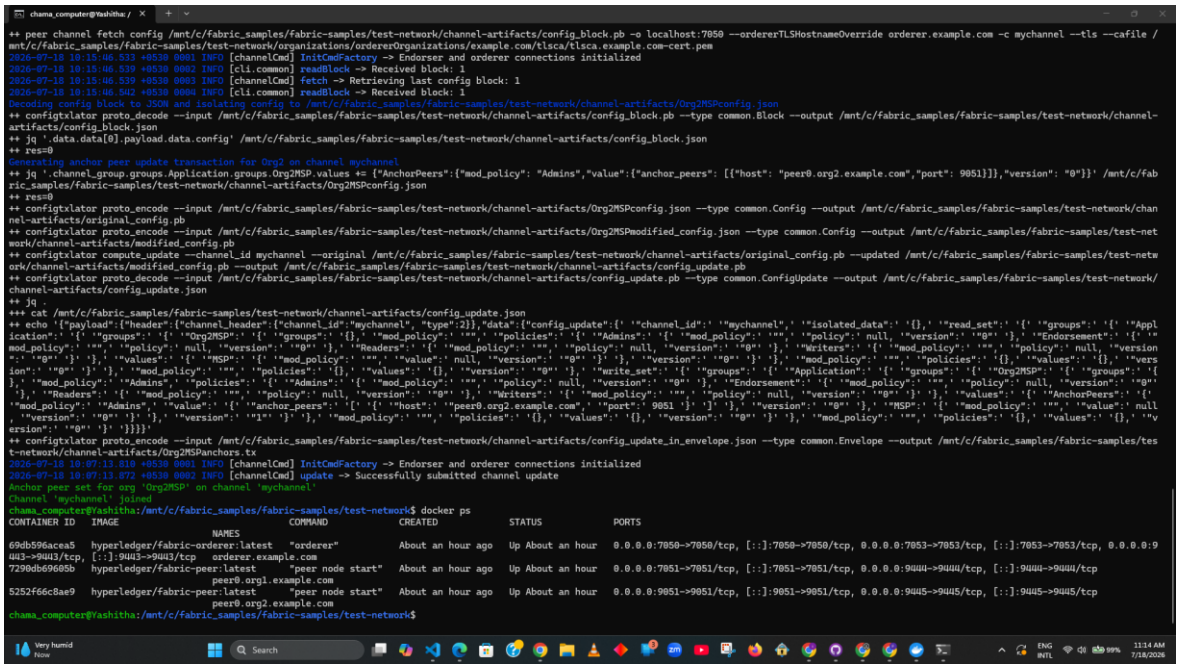


Figure 1

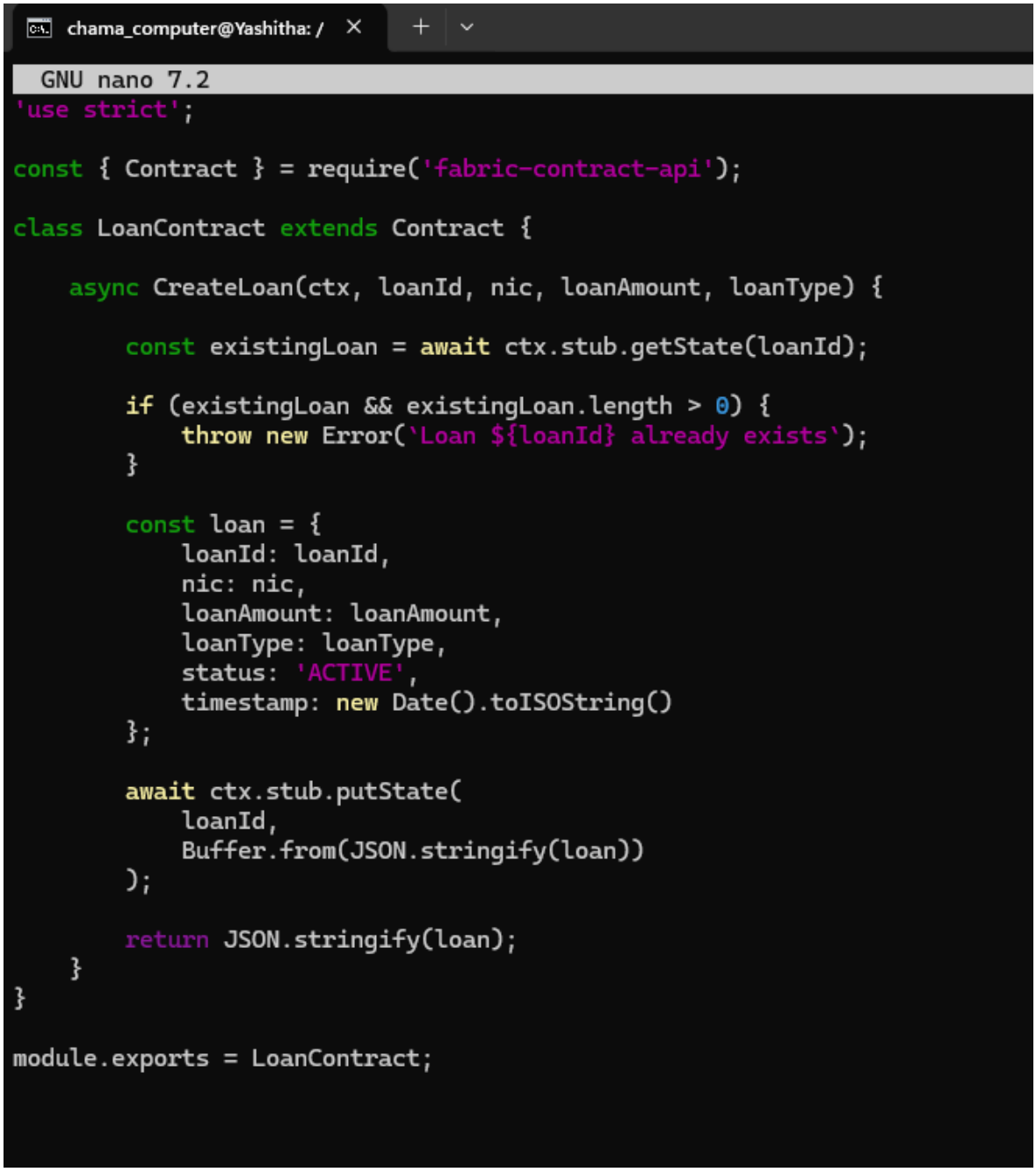

chama_computer@Yashitha: /
GNU nano 7.2
'use strict';
const { Contract } = require('fabric-contract-api');
class LoanContract extends Contract {
async CreateLoan(ctx, loanId, nic, loanAmount, loanType) {
const existingLoan = await ctx.stub.getState(loanId);
if (existingLoan && existingLoan.length > 0) {
throw new Error(`Loan ${loanId} already exists`);
}
const loan = {
loanId: loanId,
nic: nic,
loanAmount: loanAmount,
loanType: loanType,
status: 'ACTIVE',
timestamp: new Date().toISOString()
};
await ctx.stub.putState(
loanId,
Buffer.from(JSON.stringify(loan))
);
return JSON.stringify(loan);
}
}
module.exports = LoanContract;

Figure 2

.

Appendix E: Detailed Requirements Traceability Matrix

The matrix distinguishes implementation artefacts, observed technical outcomes and behaviours not demonstrated by the recorded execution. It is included as an evaluation record rather than a list of unfinished authoring tasks

Table 2

ID	Requirement	Design / implementation	Test	Evidence expected	Status
FR-01	Create a loan record from loanId, NIC, loanAmount and loanType	CreateLoan chaincode	TC-01	Implementation described; final end-to-end evidence must be attached	Implemented in design
FR-02	Reject a duplicate loanId rather than overwrite the existing record	getState existence check	TC-02	Chaincode source and duplicate rejection output	Implemented in chaincode;
FR-03	Store status as a chaincode-defined field	Record construction	TC-03	Query output from committed record	Implemented in design; verify final state
FR-04	Provide a timestamp field without endorsement mismatch	Timestamp construction	TC-04	Successful invoke endorsed by all required peers	Observed timestamp-related proposal-response mismatch
FR-05	Retrieve a committed record by loanId	Fabric query path	TC-05	Query output after confirmed commit	Implemented in design
FR-06	Keep detailed financial fields off-chain	Hybrid data model	TC-06	Schema review and database design evidence	Design defined
FR-07	Route frontend requests through backend/API	Backend boundary	TC-07	Integration test with request/response logs	Implemented in design

NFR-01	Deterministic endorsement	Chaincode serialization	TC-09	Matching proposal responses from required peers	Implemented in design
NFR-02	Protect sensitive data and credentials	API, database and operations	TC-10	Security review and sanitized logs	Production hardening is outside the prototype evaluation scope
NFR-03	Reproducible setup	Runbook and version record	TC-11	Repeatable clean start and command log	Partially documented
NFR-04	Graceful network error handling	Backend and frontend	TC-12	Controlled peer/orderer failure tests	Implemented in design
NFR-05	Auditability	Fabric ledger and application logs	TC-13	Transaction ID, commit status and query evidence	Implemented in design

Appendix F: Security and Privacy Threat Model

Table 3

Threat	Potential impact	Mitigation / design control	Validation
Unauthorized query	A user or institution accesses records beyond its role	Backend authorization, Fabric identity, least privilege, query audit	Role and access tests
Credential theft	Private key or enrollment secret is copied from a workstation	Protect key material, restrict filesystem access, rotate/revoke credentials	Secret-handling review
Raw NIC disclosure	Direct identifier is visible to channel members or logs	Assess pseudonymous reference, private data collection or controlled lookup	Privacy impact review
False source data	An authorized client submits incorrect loan data	Source validation, institutional controls,	Data-quality test

		audit and correction process	
Duplicate application	Same loan is submitted more than once under same key	Stable loanId, duplicate rejection, idempotent backend handling	TC-02 and TC-22
Endorsement inconsistency	Peers simulate different write sets	Deterministic chaincode and serialization; regression test	TC-15
Replay / duplicate request	Client retries after timeout	Idempotency strategy and post-timeout query	TC-21 and TC-22
Off-chain tampering	Detailed record changes without corresponding audit trail	Database access controls, audit logs, optional canonical digest	TC-29
Service outage	Peer, orderer, database or API unavailable	Monitoring, recovery runbook, retry policy, backup and restore tests	TC-16, TC-17, TC-30
Malicious insider	Authorized operator misuses access	Segregation of duties, approval policies, audit review	Governance and access review
Inconsistent correction	Wrong record cannot be silently erased from ledger	Correction transaction, reason code, authorized approval	Correction workflow design
Metadata inference	Amount, status or timing reveals financial activity	Minimise fields, restrict membership, evaluate privacy alternatives	Privacy review

Appendix G: Data Dictionary and Example Record

Table 4

Field	Storage location	Origin	Meaning / validation	Status
loanId	On-chain / key	User/API	Unique record key; stable identifier; must be unique within namespace	Required
NIC	On-chain in current prototype	User/API	Sensitive identifier; production use requires privacy review	Required in current schema
loanAmount	On-chain	User/API	Validated positive amount with canonical representation	Required
loanType	On-chain	User/API	Agreed loan category; validate against controlled values	Required
status	On-chain	Chaincode	Contract-defined status; not supplied by user	Generated
timestamp	On-chain	Chaincode/proposal context	Must be consistent across required endorsers	Generated; determinism to verify
monthly_income	Off-chain	Applicant/authorized source	Detailed financial field; validate source and currency/period	Off-chain
other_income	Off-chain	Applicant/authorized source	Supplementary income data	Off-chain
expenses	Off-chain	Applicant/authorized source	Expense information;	Off-chain

			define units and period	
savings_balance	Off-chain	Authorized source	Balance and as-of date should be defined	Off-chain
existing_loans	Off-chain	Authorized source	Detailed liability list or structured representation	Off-chain
repayment_history	Off-chain	Authorized source	Historical payment information with provenance	Off-chain
guarantor_support_count	Off-chain	Applicant/authorized source	Count of guarantor support records; define verification rules	Off-chain

Illustrative synthetic record (not a real customer record):

```
{
  "loanId": "DEMO-LOAN-001",
  "nic": "SYNTHETIC-NIC-001",
  "loanAmount": "250000.00",
  "loanType": "MICRO_LOAN",
  "status": "ACTIVE",
  "timestamp": "<verified deterministic timestamp>"
}
```

Appendix H: Reproducibility Runbook

H.1 Confirm the working directory

Use the Fabric Samples test-network directory and verify that the config and bin directories exist. Do not assume the current shell inherits variables from an earlier WSL session.

H.2 Set Fabric paths

Set PATH to the Fabric Samples bin directory and set FABRIC_CFG_PATH to the matching config directory. Record the Fabric peer version and Docker image versions in the test log.

H.3 Start the network

Start the test network and channel using the official test-network script. Confirm that the orderer and both organization peers are running before chaincode lifecycle commands.

H.4 Configure Org1

Set CORE_PEER_LOCALMSPID to Org1MSP, the Org1 admin MSP path, the TLS root certificate, and the host-published peer address. Confirm that TLS is enabled in the shell.

H.5 Configure Org2

Switch to Org2MSP, Org2 admin MSP path, Org2 TLS root certificate and the host-published peer address. Re-check the environment before approval or query commands.

H.6 Package and install

Package the chaincode with a unique package label. Record the package ID printed by the CLI. Install the package on each required peer and save the output.

H.7 Approve and commit

Approve the chaincode definition for each organization, check readiness, commit the definition, and record channel name, definition version, sequence and endorsement policy.

H.8 Execute a synthetic test

Submit a unique synthetic record, capture the transaction ID and result, then query the key. Do not use a real NIC or real customer financial information.

H.9 Verify final commitment

Use a ledger query after the invoke and retain the sanitized output. If the command times out, do not assume failure or success; investigate transaction status before retrying.

H.10 Record failures

Capture the exact error, time, shell environment, relevant peer logs and the corrective action. Redact secrets, keys, tokens and personal data.

H.11 Clean up

Stop the test network when finished. Keep source, package IDs and sanitized evidence needed for reproducibility, but never archive private keys in the thesis evidence folder.

Appendix J: Chaincode Design Listing and Review Notes

The following listing represents the previously documented CreateLoan pattern. It is included as a design reference, not as proof that the latest package completed an end-to-end transaction. The timestamp implementation must be reviewed and regression-tested because the development history recorded a proposal-response mismatch.

```
'use strict';

const { Contract } = require('fabric-contract-api');

class LoanContract extends Contract {
    async CreateLoan(ctx, loanId, nic, loanAmount, loanType) {
        if (!loanId || !nic || !loanAmount || !loanType) {
            throw new Error('All required loan fields must be provided');
        }

        const existingLoan = await ctx.stub.getState(loanId);
        if (existingLoan && existingLoan.length > 0) {
            throw new Error(`Loan ${loanId} already exists`);
        }

        // Timestamp behaviour must be verified for deterministic endorsement.
        const txTimestamp = ctx.stub.getTxTimestamp();
        const seconds = Number(txTimestamp.seconds);
        const nanos = Number(txTimestamp.nanos || 0);
        const timestamp = new Date(
            seconds * 1000 + Math.floor(nanos / 1000000)
        ).toISOString();

        const loan = {
            loanId,
            nic,
            loanAmount,
            loanType,
            status: 'ACTIVE',
            timestamp
        };
```

```
        await ctx.stub.putState(
            loanId,
            Buffer.from(JSON.stringify(loan))
        );
        return JSON.stringify(loan);
    }
}

module.exports = LoanContract;
```

Code review observations

- Validate loanAmount numerically and use a canonical representation before writing ledger state.
- Define allowed loanType values and an explicit length/character policy for loanId.
- Review raw NIC storage and avoid printing sensitive values to logs.
- Test timestamp equality across all required endorsers; do not assume deterministic endorsement without evidence.
- Ensure the backend waits for commit confirmation and verifies the committed record before returning success.
- Add authorization checks appropriate to the caller roles and the selected endorsement policy.
- Add a controlled correction or status-update workflow rather than directly overwriting records without audit context.

Appendix K: Architecture and Implementation Figures

This appendix collects the relevant previously supplied component diagrams and implementation screenshots. Whole-project figures are labelled as context; component-level figures are used to explain the blockchain work. Screenshots are historical evidence and should be refreshed if the final build differs.

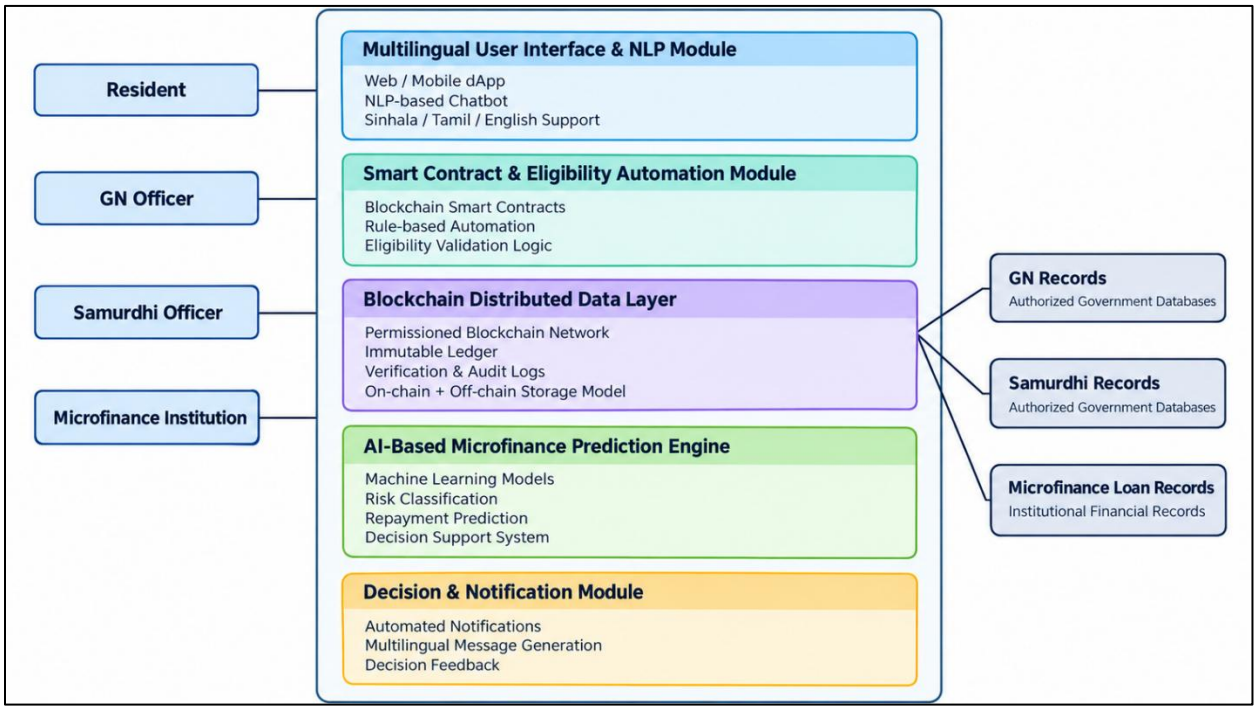

Resident
GN Officer
Samurdhi Officer
Microfinance Institution
Multilingual User Interface & NLP Module
Web / Mobile dApp
NLP-based Chatbot
Sinhala / Tamil / English Support
Smart Contract & Eligibility Automation Module
Blockchain Smart Contracts
Rule-based Automation
Eligibility Validation Logic
Blockchain Distributed Data Layer
Permissioned Blockchain Network
Immutable Ledger
Verification & Audit Logs
On-chain + Off-chain Storage Model
AI-Based Microfinance Prediction Engine
Machine Learning Models
Risk Classification
Repayment Prediction
Decision Support System
Decision & Notification Module
Automated Notifications
Multilingual Message Generation
Decision Feedback
GN Records
Authorized Government Databases
Samurdhi Records
Authorized Government Databases
Microfinance Loan Records
Institutional Financial Records

Figure 3

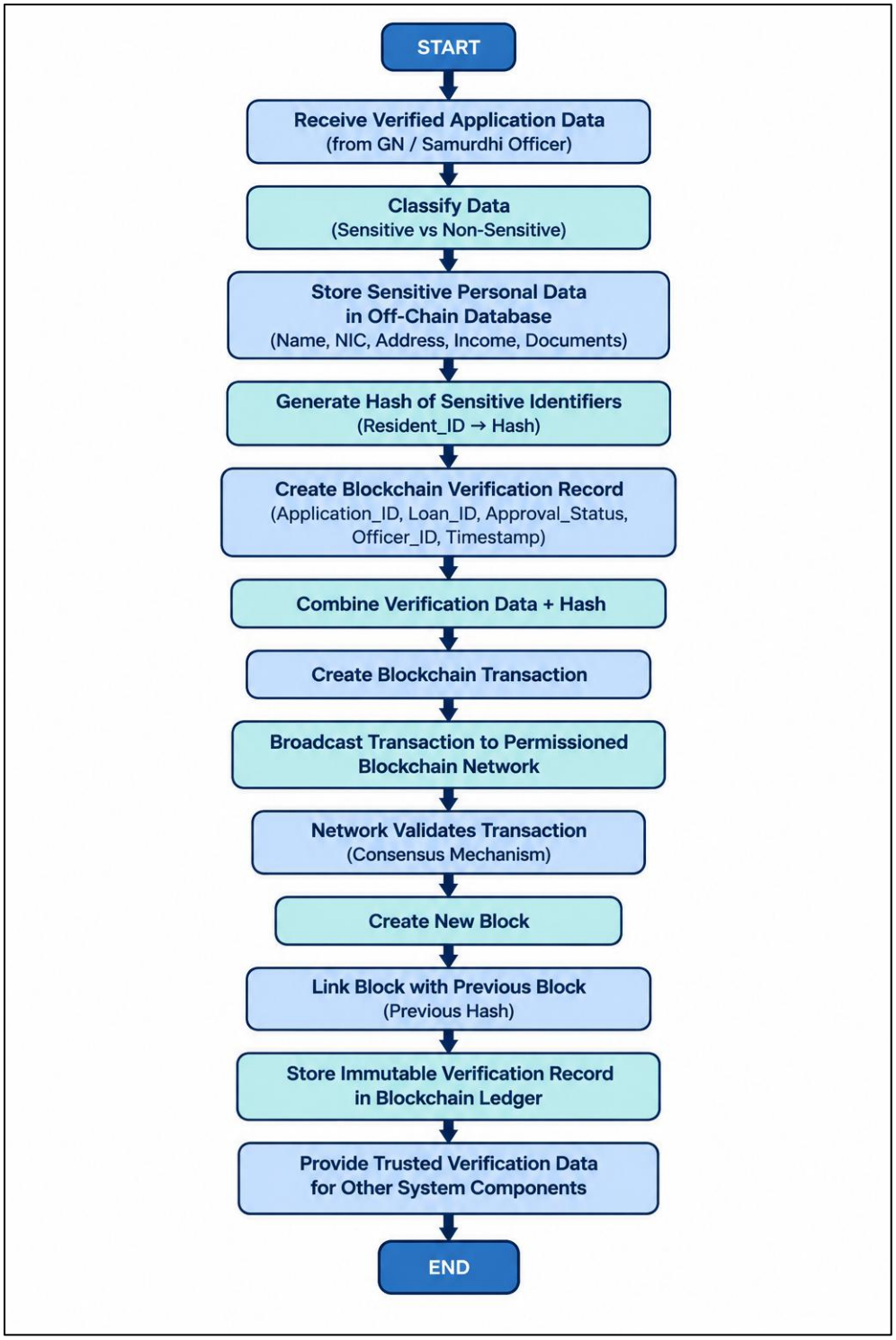

START
Receive Verified Application Data
(from GN / Samurdhi Officer)
Classify Data
(Sensitive vs Non-Sensitive)
Store Sensitive Personal Data
in Off-Chain Database
(Name, NIC, Address, Income, Documents)
Generate Hash of Sensitive Identifiers
(Resident_ID → Hash)
Create Blockchain Verification Record
(Application_ID, Loan_ID, Approval_Status,
Officer_ID, Timestamp)
Combine Verification Data + Hash
Create Blockchain Transaction
Broadcast Transaction to Permissioned
Blockchain Network
Network Validates Transaction
(Consensus Mechanism)
Create New Block
Link Block with Previous Block
(Previous Hash)
Store Immutable Verification Record
in Blockchain Ledger
Provide Trusted Verification Data
for Other System Components
END

Figure 4 .

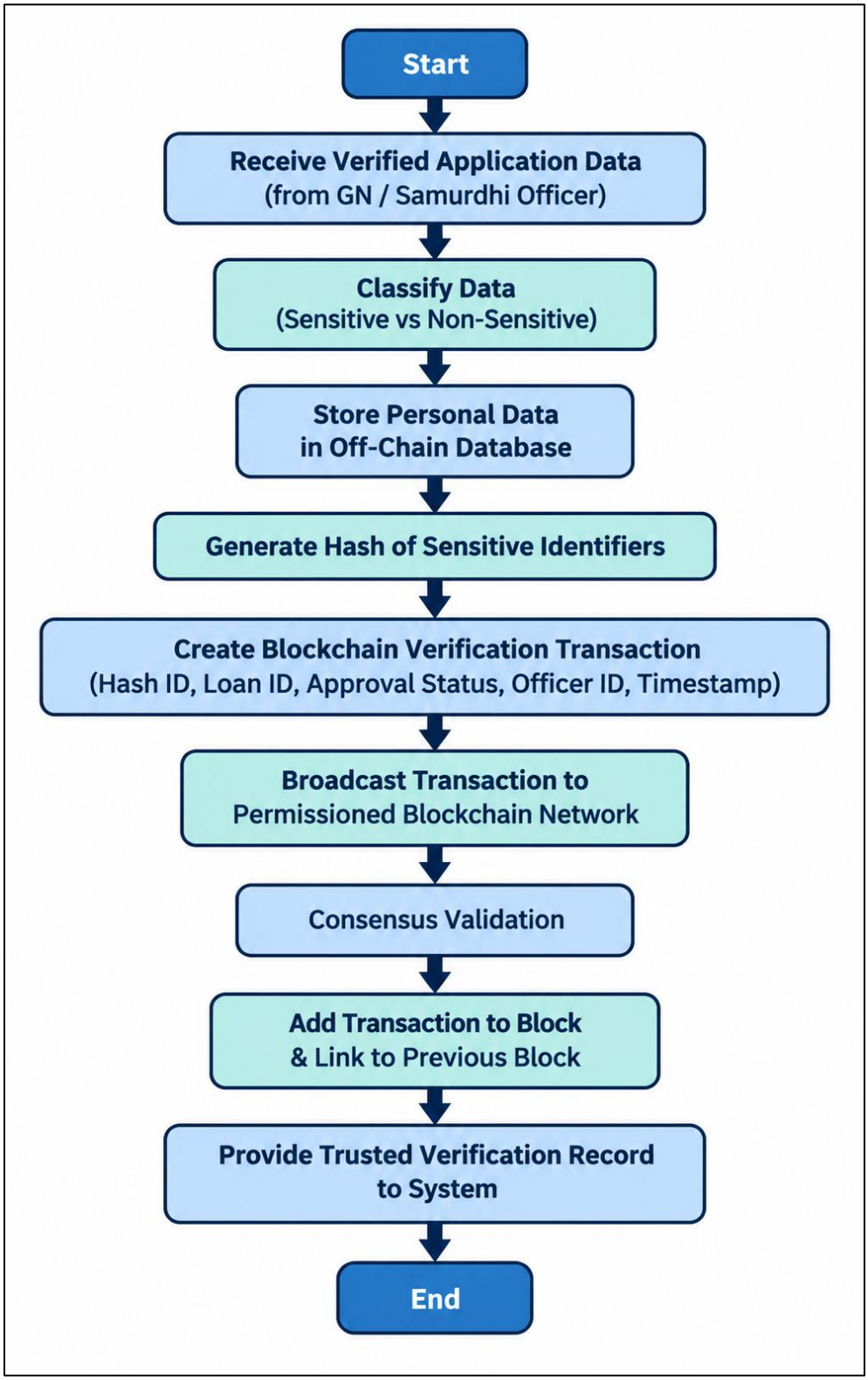

Start
Receive Verified Application Data
(from GN / Samurdhi Officer)
Classify Data
(Sensitive vs Non-Sensitive)
Store Personal Data
in Off-Chain Database
Generate Hash of Sensitive Identifiers
Create Blockchain Verification Transaction
(Hash ID, Loan ID, Approval Status, Officer ID, Timestamp)
Broadcast Transaction to
Permissioned Blockchain Network
Consensus Validation
Add Transaction to Block
& Link to Previous Block
Provide Trusted Verification Record
to System
End

Figure 5

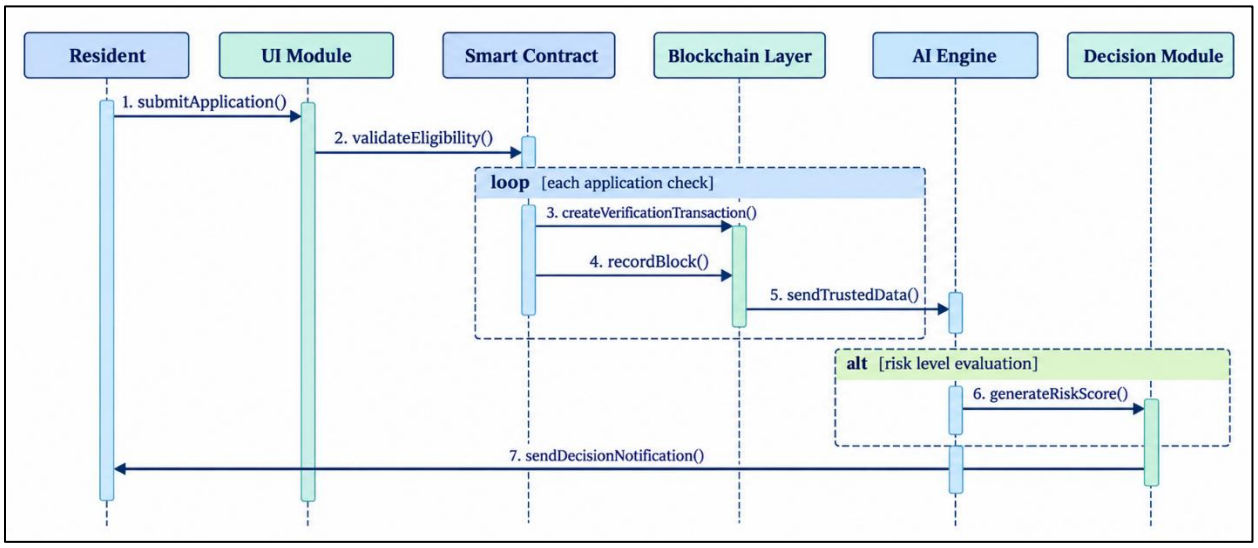

Resident
UI Module
Smart Contract
Blockchain Layer
AI Engine
Decision Module
1. submitApplication()
2. validateEligibility()
loop [each application check]
3. createVerificationTransaction()
4. recordBlock()
5. sendTrustedData()
alt [risk level evaluation]
6. generateRiskScore()
7. sendDecisionNotification()

Figure 6 .